

# Certificat de réussite

**Gana Faye**

a validé et obtenu le certificat :

**Découvrez le cloud avec Amazon Web Services**

Mathieu Nebra,  
Co-fondateur d'OpenClassrooms



Certificat n° 9730852375 - Délivré le 13 septembre 2026

**OPENCLASSROOMS**

# Découvrez le cloud avec Amazon Web Services

## Plan global du résumé AWS

### Partie 1 – EC2 (Elastic Compute Cloud)

- Types d'instances
- Tarification (On-demand, Reserved, Spot, Savings Plans, Dedicated Hosts)
- AMI, EBS, Security Groups
- Connexion SSH & IP Élastique
- Sauvegardes (AMI & Snapshots)

### Partie 2 – RDS (Relational Database Service)

- Pourquoi utiliser RDS (DBaaS vs installer sur EC2)
- Moteurs supportés (MySQL, PostgreSQL, Aurora, Oracle, SQL Server, MariaDB)
- Multi-AZ, sauvegardes automatiques & snapshots
- Restauration et scalabilité
- Cas pratiques

### Partie 3 – S3 (Simple Storage Service)

- Buckets, objets et métadonnées
- Classes de stockage (Standard, IA, Glacier, Intelligent Tiering, etc.)
- Gestion des accès (politiques IAM, stratégies de bucket)
- Cycle de vie (expiration, archivage)
- Cas pratiques (hébergement fichiers statiques, backups, CDN)

### Partie 4 – IAM (Identity and Access Management)

- Utilisateur racine vs comptes IAM
- Politiques IAM vs stratégies de ressource
- Rôles IAM, groupes et permissions
- MFA, bonnes pratiques de sécurité
- Cas pratiques (ex. : donner accès lecture seule à un bucket S3)

## Partie 5 – Billing (Facturation & optimisation des coûts)

- Tableau de bord Billing
- Cost Explorer : analyse par service, balises, ressources
- Savings Plans (EC2)
- Instances réservées (EC2/RDS)
- Optimisation S3 (Intelligent Tiering, cycle de vie)
- Astuces pour réduire les coûts

## Partie 6 – Glossaire complet

- Tous les mots-clés (EC2, EBS, AMI, RDS, VPC, IAM, Policy, S3, Bucket, Glacier, etc.)
- Définitions courtes, simples et claires

## Partie 1 – Amazon EC2 (Elastic Compute Cloud)

Amazon EC2 est l'un des services **fondamentaux** d'AWS. Il permet de créer et gérer des machines virtuelles dans le cloud, appelées **instances**. L'idée est simple : plutôt que d'acheter et gérer vos propres serveurs physiques, vous "louez" des serveurs virtuels à la demande, facturés à la seconde ou à l'heure.

C'est la brique de base pour héberger des applications, sites web, bases de données, ou même des environnements de test et développement.

### 1. Types d'instances EC2

Les instances EC2 sont classées en **familles**, chacune adaptée à un type de charge de travail :

- **Usage général (t2.micro, t3.small, m5.large, etc.)**  
Équilibre entre CPU, mémoire et stockage.  
Utilisation : sites web, applications légères, environnements de développement.  
Exemple : **t2.micro** (gratuite 1 an dans l'offre Free Tier).
- **Calcul optimisé (c5, c6g, etc.)**  
Beaucoup de puissance CPU.  
Utilisation : calcul scientifique, analyse de données, rendu, serveurs web haute performance.
- **Mémoire optimisée (r5, x1, z1d, etc.)**  
Grande quantité de RAM.

Utilisation : bases de données en mémoire (Redis), gros caches, applications analytiques.

- **Stockage optimisé (i3, d2, etc.)**

Disques NVMe très rapides.

Utilisation : Big Data, Hadoop, bases de données nécessitant beaucoup d'I/O.

- **Accélération matérielle (p3, g4, inf1, etc.)**

Accélérateurs comme GPU ou FPGA.

Utilisation : Machine Learning, rendu 3D, IA, reconnaissance vocale.

**À retenir** : pour débiter, on choisit en général **t2.micro** (usage général, gratuite). Pour des besoins spécifiques, on adapte le type d'instance.

## 2. Tarification des instances EC2

EC2 fonctionne sur un modèle **Pay-as-you-go** (paiement à l'usage). Plusieurs options existent :

### 1. À la demande (On-Demand)

- Pas d'engagement, paiement à l'heure ou à la seconde.
- Flexibilité maximale, mais le coût est plus élevé.  
Idéal pour les tests, projets temporaires, débutants.

### 2. Instances réservées (Reserved Instances)

- Engagement sur 1 ou 3 ans.
- Jusqu'à **72 % de réduction** par rapport au prix à la demande.  
Idéal pour les charges de travail stables et prévisibles.

### 3. Instances Spot

- Instances très bon marché (jusqu'à 90 % de réduction).
- AWS peut interrompre l'instance à tout moment si la capacité est reprise.  
Idéal pour les tâches flexibles et non critiques (ex. rendu vidéo, simulations).

### 4. Savings Plans

- Alternative flexible aux instances réservées.
- Vous vous engagez sur une **consommation horaire minimale** (ex. \$0.10/h) pendant 1 ou 3 ans.

- Réduction d'environ 30-50 %.  
Idéal pour des environnements stables, avec plus de souplesse que les instances réservées.

## 5. Hôtes dédiés

- Vous louez un serveur physique complet.
- Utilisé pour des licences logicielles spécifiques (Windows, Oracle) ou des besoins de conformité.

## 3. AMI (Amazon Machine Image)

Pour lancer une instance EC2, vous devez choisir une **AMI** (image machine). Une AMI est une **image préconfigurée** qui contient un système d'exploitation et éventuellement des logiciels.

- **AMI de base :**

- Amazon Linux (optimisé AWS, gratuit)
- Ubuntu Server
- Red Hat Enterprise Linux
- SUSE Linux
- Windows Server

- **AMI Marketplace :**

- Images préconfigurées avec logiciels (LAMP stack, WordPress, etc.).
- Certaines gratuites, d'autres payantes.

Exemple : utiliser une AMI "Bitnami LAMP" permet de démarrer directement un serveur Apache + PHP + MySQL sans configuration manuelle.

## 4. EBS (Elastic Block Store)

Chaque instance EC2 utilise un disque appelé **volume EBS**.

- Par défaut, un disque de 8–30 Go SSD est fourni (10 Go gratuits dans le Free Tier).
- Plusieurs volumes peuvent être attachés à une instance.
- Fonctionnalités :
  - **Snapshots (sauvegardes)**
  - **Chiffrement**

- **Elasticité** (on peut augmenter la taille sans recréer le disque).

Différence clé : l'instance = puissance de calcul, EBS = stockage persistant.

## 5. Security Groups

Un **Security Group** est un **pare-feu virtuel** associé à une instance EC2.

Il définit **qui peut accéder** à votre machine et sur quels ports.

Exemples de règles courantes :

- **SSH (22/tcp)** : accès à distance pour administrer le serveur.
- **HTTP (80/tcp)** et **HTTPS (443/tcp)** : accès web.
- Restreindre SSH uniquement à votre **IP personnelle** pour la sécurité.

Bonnes pratiques :

- Toujours limiter l'accès SSH (pas de 0.0.0.0/0).
- Utiliser plusieurs Security Groups pour séparer les rôles (ex. un pour Web, un pour BDD).

## 6. Connexion SSH & Clés

- Lors de la création d'une instance, AWS vous demande de générer une **clé SSH (.pem)**.
- Cette clé permet de vous connecter sans mot de passe.
- Commande de connexion (exemple avec Ubuntu) :
- `chmod 400 ~/.ssh/ma_cle.pem`
- `ssh -i ~/.ssh/ma_cle.pem ubuntu@ec2-xx-xx-xx.compute.amazonaws.com`

Si vous perdez la clé, vous ne pouvez plus vous connecter à l'instance.

## 7. IP Élastique

- Par défaut, une instance EC2 obtient une **IP publique dynamique**.
- Si elle redémarre, l'IP change.
- Solution : attacher une **IP Élastique** (Elastic IP).
- Une Elastic IP est une **adresse IP fixe** que vous associez à une instance.
- Gratuite tant qu'elle est attachée à une instance active.

Indispensable pour un site web en production (stabilité de l'adresse).

## 8. Sauvegardes & Restauration

Deux méthodes principales :

### 1. AMI (Amazon Machine Image)

- Sauvegarde complète (OS + configuration + données).
- Permet de recréer une instance identique.
- Rapide mais coûte plus de stockage.

### 2. Snapshots EBS

- Sauvegarde incrémentielle du disque.
- Économique (seules les différences sont enregistrées).
- Pour restaurer : créer un nouveau volume à partir du snapshot et l'attacher à une instance.

Bonnes pratiques :

- Faire un snapshot avant chaque mise à jour critique.
- Nettoyer régulièrement les vieilles AMI et snapshots pour réduire les coûts.

## En résumé (EC2)

- **EC2 = machines virtuelles à la demande dans le cloud.**
- **Types d'instances** : Usage général, calcul, mémoire, stockage, GPU.
- **Tarification** : On-demand, Reserved, Spot, Savings Plans.
- **AMI** : images préconfigurées pour démarrer plus vite.
- **EBS** : stockage persistant attaché aux instances.
- **Security Groups** : firewall virtuel contrôlant l'accès.
- **Clés SSH** : authentification sécurisée.
- **IP Élastique** : IP fixe pour vos serveurs.
- **Snapshots/AMI** : sauvegardes pour restaurer rapidement vos environnements.

## Partie 2 – Amazon RDS (Relational Database Service)

Amazon RDS est le service **DBaaS** (Database-as-a-Service) d'AWS. Il permet de lancer et gérer une base de données relationnelle sans avoir à se soucier de l'installation, des patchs, de la maintenance matérielle ou des sauvegardes. Là où EC2 vous laisse une grande liberté mais vous impose de tout gérer, RDS délègue la complexité à AWS pour vous concentrer sur vos applications.

### 1. Pourquoi utiliser Amazon RDS ?

Sur une instance **EC2**, vous pouvez très bien installer MySQL ou PostgreSQL. Mais cela implique de gérer :

- l'installation du moteur de base de données ;
- les patchs de sécurité et mises à jour ;
- la configuration fine (buffers, paramètres de performance) ;
- les sauvegardes et restaurations manuelles ;
- la réplication et la haute disponibilité.

Avec **RDS**, AWS s'occupe de :

- déployer automatiquement un moteur SQL prêt à l'emploi,
- appliquer les mises à jour logicielles et de sécurité,
- planifier et exécuter les sauvegardes automatiques,
- permettre une montée en charge verticale (changer de type d'instance) ou horizontale (réplication),
- surveiller la santé de l'instance via CloudWatch.

En résumé : **moins de gestion système, plus de focus sur les données et les applications.**

### 2. Moteurs de bases de données supportés

RDS supporte la plupart des moteurs SQL populaires :

- **MySQL** – le moteur open source le plus utilisé, supporté par d'innombrables applications web (WordPress, Joomla, ERP, etc.).
- **MariaDB** – fork de MySQL, souvent préféré pour ses performances et sa licence 100 % open source.
- **PostgreSQL** – base relationnelle avancée avec support JSON, très robuste pour les applications modernes.

- **Oracle Database** – moteur propriétaire, utilisé dans de nombreuses grandes entreprises.
- **Microsoft SQL Server** – base relationnelle intégrée à l'écosystème Windows.
- **Amazon Aurora** – moteur propriétaire AWS, compatible MySQL et PostgreSQL, annoncé comme 3 à 5 fois plus rapide, entièrement optimisé pour le cloud.

Pour les projets classiques (sites web, applications métier), **MySQL** et **PostgreSQL** restent les choix les plus courants.

### 3. Création d'une instance RDS (exemple avec MySQL)

1. **Aller dans la console RDS** → “Créer une base de données”.
2. **Choisir le moteur** (ex. MySQL).
3. **Sélectionner le mode** :
  - **Développement/Test** → petit stockage, pas de réplication, idéal pour apprendre ou tester.
  - **Production** → inclut par défaut la haute disponibilité multi-AZ, plus de stockage et sauvegardes renforcées.
  - **Free Tier** → db.t3.micro (1 vCPU, 1 Go RAM, 20 Go de stockage).
4. **Configurer l'instance** :
  - Identifiant de la DB (nom de l'instance),
  - Nom d'utilisateur administrateur,
  - Mot de passe administrateur.
5. **Choisir le type d'instance** → CPU et mémoire (ex. db.t3.micro ou db.m5.large).
6. **Stockage** : SSD général (gp2/gp3), SSD provisionné (IOPS garantis), ou magnétique standard.
7. **Paramètres réseau (VPC, sous-réseaux, accessibilité publique)** :
  - Pour un usage sécurisé → instance privée dans le même VPC que votre application.
  - Pour tester → vous pouvez activer l'accessibilité publique (avec prudence !).
8. **Sauvegardes automatiques** → activées par défaut (rétention 7 jours, configurable jusqu'à 35 jours).

9. **Surveillance avancée** → intégrée avec CloudWatch Metrics (CPU, connexions, requêtes lentes).

10. **Créer la base de données.**

Une fois lancée, AWS vous fournit un **endpoint DNS** (ex. mydb.abc123.eu-west-3.rds.amazonaws.com:3306). C'est cette adresse que vos applications utiliseront pour se connecter.

#### 4. Sécurité et bonnes pratiques avec RDS

- **IAM** : contrôler qui peut créer/supprimer/configurer des instances RDS.
- **Security Groups** : définir quels serveurs (ex. vos EC2) peuvent accéder au port 3306 (MySQL) ou 5432 (PostgreSQL).
- **Pas d'accès root SSH** : contrairement à EC2, vous n'avez pas accès au système d'exploitation de l'instance. Vous ne pouvez agir que via SQL et la console AWS.
- **Chiffrement** : possibilité d'activer le chiffrement au repos (KMS) et en transit (SSL/TLS).
- **Pas d'exposition publique inutile** : privilégier l'accès privé via VPC.

#### 5. Sauvegardes et restauration

- **Sauvegardes automatiques** : journaux binaires + snapshot quotidien → permet la **restauration à un instant précis** (point-in-time recovery).
- **Instantanés manuels (Snapshots)** : créés par l'utilisateur avant une migration ou une opération risquée.
- **Restauration** : un snapshot restaure une **nouvelle instance RDS** avec un nouvel endpoint. Vous pouvez ensuite réassocier vos applications.

Exemple : avant une mise à jour d'application, vous créez un snapshot. Si un bug supprime des données, vous restaurez le snapshot en quelques minutes.

#### 6. Haute disponibilité et scalabilité

- **Multi-AZ** : RDS réplique automatiquement la base dans une autre zone de disponibilité (datacenter distinct). Si le serveur principal tombe, la réplique prend le relais → **haute disponibilité garantie**.
- **Read Replicas** : copies en lecture seule de la base principale, utiles pour répartir la charge (ex. séparer les requêtes SELECT lourdes).

- **Scaling vertical** : changer de type d'instance pour ajouter plus de CPU ou RAM.
- **Scaling horizontal** : ajouter des replicas pour absorber plus de trafic.

## 7. Tarification de RDS

Vous payez :

- **Instance** : type (t3.micro, db.m5.large, etc.), nombre de vCPUs et RAM.
- **Stockage** : SSD ou HDD, taille et provisionnement IOPS.
- **Sauvegardes** : stockage gratuit jusqu'à la taille totale de l'instance DB (ex. instance 100 Go = 100 Go de backups gratuits).
- **Transfert de données** : gratuit dans un même VPC, facturé si sortie d'AWS.

Réductions :

- **Instances réservées** (1 ou 3 ans) → jusqu'à 70 % d'économie.
- **Arrêt/relance** d'instances en Dev/Test pour payer uniquement quand elles sont utilisées.

## 8. Cas pratiques

- **Site web WordPress** :
  - EC2 → héberge Apache/PHP.
  - RDS → héberge MySQL pour les articles, utilisateurs, etc.
  - S3 → stocke les images des articles.
- **Application e-commerce** :
  - RDS PostgreSQL → gère les commandes, clients, inventaires.
  - Multi-AZ activé pour garantir la disponibilité.
  - Réplica en lecture pour le reporting et les tableaux de bord.
- **Sauvegardes critiques** :
  - RDS → sauvegardes automatiques activées.
  - Snapshots exportés vers **S3 Glacier** pour l'archivage long terme.

## En résumé (RDS)

- **RDS = DBaaS** : base de données managée, sans gestion système.
- Supporte les principaux moteurs (MySQL, MariaDB, PostgreSQL, Oracle, SQL Server, Aurora).
- **Avantages** : patches automatiques, sauvegardes, monitoring, scaling facile.
- **Sécurité** : IAM, Security Groups, VPC privé, chiffrement.
- **Haute disponibilité** : Multi-AZ, Read Replicas.
- **Restauration** : snapshots automatiques/manuels, point-in-time recovery.
- **Économies** : instances réservées, bonne gestion du stockage et des classes S3 pour l'archivage.

## Partie 3 – Amazon S3 (Simple Storage Service)

Amazon **S3** est l'un des services les plus populaires d'AWS. Il fournit un système de **stockage objet** hautement disponible, évolutif et sécurisé. Contrairement à un disque classique (EBS ou HDD), où vous gérez un système de fichiers, S3 stocke des objets (fichiers + métadonnées) dans des **buckets** accessibles via une URL ou une API.

Ce service est largement utilisé pour stocker des images, des vidéos, des sauvegardes, des archives et même pour héberger des sites statiques.

### 1. Pourquoi utiliser Amazon S3 ?

- **Évolutivité quasi infinie** : vous pouvez stocker des milliards de fichiers, sans limite de capacité.
- **Durabilité garantie (11 9s : 99.999999999 %)** : grâce à la réplication automatique sur plusieurs datacenters d'une même région.
- **Coûts réduits** : jusqu'à 4 fois moins cher qu'EBS, avec des classes de stockage adaptées à différents usages.
- **Accès global** : les fichiers peuvent être accessibles depuis n'importe où via HTTPS.
- **Sécurité intégrée** : IAM, politiques de bucket, chiffrement au repos et en transit.
- **Intégration avec d'autres services AWS** : sauvegardes RDS, logs CloudTrail, machine learning, etc.

En bref : **S3 = un entrepôt universel pour vos données.**

## 2. Structure d'Amazon S3

### 1. Buckets :

- Ce sont des conteneurs logiques qui regroupent vos objets.
- Chaque bucket a un nom unique **dans toute AWS** (ex. mon-bucket-images-2025).
- Un bucket est créé dans une région donnée (ex. eu-west-3 à Paris).

### 2. Objets :

- Chaque fichier stocké dans S3 est un objet.
- Un objet = données binaires (fichier) + métadonnées (auteur, type MIME, tags).
- Chaque objet est identifié par une clé (chemin/fichier.jpg).

### 3. Clés et préfixes :

- Il n'y a pas de répertoire physique comme sur un disque dur. Les "dossiers" sont en réalité des préfixes dans les clés.
- Exemple : backups/2025/janvier/db.sql → S3 interprète backups/2025/janvier/ comme un chemin, mais il s'agit simplement d'une chaîne de caractères dans la clé.

## 3. Création et gestion d'un bucket

### Étapes principales :

#### 1. Créer un bucket

- Nom global unique.
- Choisir une région proche de vos utilisateurs (latence plus faible).
- Configurer les options (versioning, chiffrement, logs, balises).

#### 2. Charger un fichier (upload)

- Depuis la console AWS : bouton *Charger*.
- Depuis la CLI :
- `aws s3 cp monfichier.jpg s3://mon-bucket/`

#### 3. Télécharger un fichier (download)

- Console AWS : clic droit → *Télécharger*.

- CLI :
- `aws s3 cp s3://mon-bucket/monfichier.jpg ./local/`

#### 4. Gestion des droits

- Par défaut, les objets sont **privés**.
- L'accès public se configure via des **bucket policies** (JSON), ou via un lien pré-signé généré par l'application.

#### 4. Sécurité et contrôle d'accès

S3 permet de contrôler l'accès avec plusieurs couches :

- **IAM Policies** → définissent les actions qu'un utilisateur/role AWS peut faire (upload, delete, list).
- **Bucket Policies** → définissent ce que tout utilisateur (ou un groupe) peut faire sur un bucket donné.
- **ACL (Access Control Lists)** → héritées des débuts de S3, utilisées de manière plus rare aujourd'hui.
- **Public Access Block** → option de sécurité par défaut qui empêche les buckets d'être publics par erreur.

Exemple de bucket policy qui rend publiques toutes les images d'un bucket :

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::mon-bucket-images/*"
    }
  ]
}
```

Cela permet d'héberger des fichiers statiques accessibles à tous (images de site web, fichiers CSS, etc.).

## 5. Classes de stockage dans S3

AWS propose plusieurs niveaux de stockage selon vos besoins :

### 1. S3 Standard

- Usage général (fichiers utilisés régulièrement).
- Coût : ~\$0.023/Go/mois.
- Latence faible (millisecondes).

### 2. S3 Standard-IA (Infrequent Access)

- Pour des données peu consultées mais nécessitant un accès rapide.
- Moins cher (~\$0.0125/Go/mois).

### 3. S3 Intelligent-Tiering

- Déplace automatiquement les fichiers entre Standard et IA selon leur utilisation.
- Idéal pour optimiser sans réfléchir.

### 4. S3 Glacier Instant Retrieval

- Données rarement utilisées, mais récupération en quelques minutes.
- ~80 % moins cher que Standard.

### 5. S3 Glacier Deep Archive

- Archivage long terme (conservation légale, backups historiques).
- Ultra bon marché (~\$0.00099/Go/mois).
- Temps de récupération : plusieurs heures.

Bonne pratique : **mélanger les classes de stockage**. Exemple :

- Images de site = Standard.
- Backups mensuels = Glacier.
- Logs = Intelligent-Tiering.

## 6. Gestion du cycle de vie

Pour éviter de remplir vos buckets avec des milliers de fichiers inutiles, vous pouvez définir des **Lifecycle Rules** :

- **Exemple 1** : déplacer tous les fichiers backups/ vers Glacier après 30 jours.
- **Exemple 2** : supprimer automatiquement les logs après 180 jours.

- **Exemple 3** : activer le versioning et supprimer les anciennes versions après 90 jours.

Ces règles se configurent dans l'onglet **Gestion** du bucket.

## 7. Cas d'utilisation concrets

### 1. Hébergement d'images de sites web

- Stocker toutes les images sur S3.
- Configurer un accès public en lecture.
- Utiliser CloudFront (CDN d'AWS) pour accélérer la diffusion mondiale.

### 2. Sauvegardes de bases de données

- Exporter les dumps MySQL/PostgreSQL vers S3.
- Déplacer automatiquement en Glacier via une règle de cycle de vie.

### 3. Archivage légal

- Pour les banques, assurances, hôpitaux → conservation obligatoire de données.
- Stockage économique via Glacier Deep Archive.

### 4. Big Data & Machine Learning

- Stocker des datasets bruts sur S3.
- Consommer directement depuis EMR, SageMaker, ou Lambda.

### 5. Static Website Hosting

- Héberger un site statique (HTML, CSS, JS) directement depuis un bucket configuré en mode "Website Hosting".
- Coupler avec Route53 pour pointer un domaine.

## 8. Tarification

La facturation S3 repose sur plusieurs facteurs :

- **Stockage mensuel (Go)** → dépend de la classe choisie.
- **Nombre de requêtes** → PUT, GET, DELETE.
- **Bande passante sortante** → vers Internet, facturée au Go.
- **Fonctionnalités avancées** → réplication inter-régions, CloudFront, stockage Intelligent-Tiering.

Exemple :

- 100 Go d'images en Standard = \$2.30/mois.
- 1 To de backups en Glacier = ~\$4.50/mois.

### ✚ En résumé (S3)

- **S3 = stockage objet** → fichiers (objets) organisés dans des buckets.
- **Haute durabilité et disponibilité** → données répliquées sur plusieurs datacenters.
- **Sécurité** → IAM, bucket policies, chiffrement, blocage de l'accès public.
- **Classes de stockage** → Standard, IA, Intelligent-Tiering, Glacier, Deep Archive.
- **Optimisation des coûts** → lifecycle rules + choix intelligent des classes.
- **Cas d'usage** : hébergement d'images, sauvegardes, archivage légal, sites statiques, Big Data.

## Partie 4 – IAM (Identity and Access Management)

IAM est le service **central** d'AWS pour gérer la sécurité, les identités et les droits d'accès. C'est grâce à lui que vous contrôlez **qui** peut agir sur **quoi** dans vos ressources cloud. C'est un service invisible mais absolument vital, car une mauvaise configuration peut exposer tout votre environnement.

### 1. Rôle fondamental d'IAM

- **Authentification** : gérer l'identité des utilisateurs (humains ou machines).
- **Autorisation** : définir ce qu'ils peuvent ou ne peuvent pas faire.
- **Gestion fine des accès** : appliquer le principe du **moindre privilège** (ne donner que les droits nécessaires).
- **Traçabilité** : toutes les actions passent par IAM et sont journalisées (CloudTrail).

Sans IAM bien configuré, vous prenez le risque de fuites de données, d'accès non autorisés, voire de suppression totale de votre infrastructure.

## 2. Les entités principales d'IAM

### 1. Utilisateur IAM

- Représente une personne physique (ex. Alice, Bob, David).
- Chaque utilisateur peut avoir :
  - un accès **console** (login + mot de passe)
  - un accès **programmatique** (via des clés d'API).
- Exemple : un développeur accède à la console AWS pour créer un bucket.

### 2. Groupes IAM

- Permettent de regrouper plusieurs utilisateurs pour leur appliquer des politiques communes.
- Exemple : le groupe Développeurs peut avoir accès à S3 et EC2, mais pas à la facturation.

### 3. Rôles IAM

- Ce sont des identités temporaires **sans mot de passe ni clé permanente**.
- Les rôles sont souvent utilisés par des services AWS ou des applications.
- Exemple : une instance EC2 prend le rôle S3-Access pour sauvegarder automatiquement ses logs dans S3.

### 4. Politiques IAM (Policies)

- Documents JSON qui définissent les permissions (autoriser ou refuser des actions).
- Elles peuvent être attachées à un utilisateur, un groupe ou un rôle.
- Exemple : une politique AmazonS3ReadOnlyAccess donne uniquement le droit de lire les objets S3.

### 3. Structure d'une politique IAM

Une policy IAM est un document JSON. Exemple d'une politique qui donne accès en lecture seule à un bucket :

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:ListBucket",
        "s3:GetObject"
      ],
      "Resource": [
        "arn:aws:s3:::mon-bucket",
        "arn:aws:s3:::mon-bucket/*"
      ]
    }
  ]
}
```

#### Explication :

- **Effect** : autoriser (Allow) ou interdire (Deny).
- **Action** : liste des actions permises (ex. lire un objet, lister un bucket).
- **Resource** : ressource concernée, identifiée par son ARN (Amazon Resource Name).

**Le principe d’AWS** : tout est interdit par défaut. Une action n’est permise que si une policy l’autorise explicitement.

### 4. Différence entre IAM et Bucket Policies (exemple avec S3)

- **Policy IAM** → centrée sur l'utilisateur (ex. Bob peut lire S3).
- **Bucket Policy** → centrée sur la ressource (ex. tout Internet peut lire le bucket public-images).

- **ACL (Access Control List)** → ancienne méthode, encore supportée mais peu utilisée.

En cas de conflit entre une policy IAM et une bucket policy, c'est **l'interdiction la plus stricte** qui l'emporte.

## 5. Bonnes pratiques de sécurité avec IAM

1. **Ne jamais utiliser le compte racine** sauf pour les tâches critiques (ex. configuration initiale).
2. **Activer MFA (multi-factor authentication)** pour le compte root et les utilisateurs sensibles.
3. **Appliquer le principe du moindre privilège** : donner uniquement les droits nécessaires.
4. **Utiliser des rôles IAM** pour les services (ex. EC2 → accès à S3), plutôt que des clés stockées en dur.
5. **Éviter les accès publics non contrôlés** (ex. bucket S3 ouvert à tous par erreur).
6. **Suivre et auditer les accès** avec AWS CloudTrail.
7. **Utiliser des politiques gérées par AWS** pour débiter, puis affiner avec des politiques personnalisées.

## 6. Exemples concrets d'utilisation

### Exemple 1 – Développeur en lecture seule sur S3

- Créer un utilisateur IAM dev\_readonly.
- Lui attacher la policy AmazonS3ReadOnlyAccess.
- Il pourra télécharger des fichiers depuis les buckets, mais pas en ajouter ou supprimer.

### Exemple 2 – Application sur EC2 sauvegardant des logs

- Créer un rôle IAM S3-Logs-Writer.
- Lui attacher une policy s3:PutObject sur un bucket précis.
- Attacher ce rôle à l'instance EC2 → l'application peut écrire dans S3 sans clé API.

### Exemple 3 – Partage avec un partenaire externe

- Créer une bucket policy qui autorise uniquement le compte AWS du partenaire (Principal).
- L'accès est limité aux fichiers d'un dossier (ex. exports/).
- Le partenaire configure une policy IAM côté utilisateur pour compléter l'accès.

## 7. Intégration IAM avec les autres services AWS

- **EC2** : rôle IAM pour accéder à S3 ou RDS sans clé d'API.
- **RDS** : gestion des connexions via IAM Database Authentication.
- **S3** : bucket policies pour ouvrir l'accès public ou intercomptes.
- **Lambda** : chaque fonction Lambda utilise un rôle IAM pour interagir avec d'autres services.
- **Billing/Cost Explorer** : contrôle de qui peut consulter ou modifier les coûts.

## 8. En résumé (IAM)

- IAM est le **système nerveux de la sécurité AWS**.
- Il gère les **utilisateurs, groupes, rôles et politiques**.
- Les politiques IAM sont des documents JSON qui autorisent ou interdisent des actions.
- Toujours appliquer le **moindre privilège**.
- Utiliser **MFA**, éviter le **compte racine**, et préférer les **rôles IAM** aux clés statiques.
- IAM s'intègre à tous les services AWS pour contrôler finement les accès.

## Partie 5 – Billing et Optimisation des coûts AWS

Dans AWS, la facturation n'est pas un simple outil administratif : c'est un véritable **levier stratégique**. Comme le cloud fonctionne en **pay-as-you-go** (paiement à l'usage), comprendre et optimiser vos coûts peut représenter un avantage concurrentiel majeur.

## 1. Principes de la facturation AWS

### Fonctionnement général

- Vous payez uniquement ce que vous consommez : CPU, RAM, stockage, bande passante, etc.
- La facturation est calculée **à l'heure** ou **à la seconde** selon les services.
- Trois variables influencent la facture :
  - . **Le type de ressource** (EC2, RDS, S3, etc.)
  - . **La région** (certains datacenters coûtent plus cher, ex. Paris > Virginie)
  - . **Le mode de facturation** (à la demande, savings plan, instances réservées...).

### Pourquoi c'est critique

- Dans une petite startup, réduire la facture de 30 % peut libérer du budget pour recruter ou investir.
- Dans une grande entreprise, la différence se compte en **milliers voire millions de dollars** par an.

## 2. Console de facturation (Billing Dashboard)

Accessible en cliquant sur le nom du compte puis **Tableau de bord de facturation**.

### Sections principales

- **Factures** : détail précis des coûts par service, région et type d'usage.
- **Paielements** : historique des règlements effectués.
- **Cost Explorer** : l'outil phare pour analyser et prévoir les coûts.
- **Budgets** : permet de fixer des seuils d'alerte (mail ou SMS) quand on dépasse un certain montant.

Dès la création du compte, il est recommandé d'activer les **alertes de facturation CloudWatch**, afin d'éviter les mauvaises surprises (ex. instances oubliées allumées).

## 3. Cost Explorer – l'outil d'analyse des coûts

### Fonctionnalités

- **Graphiques dynamiques** : suivre l'évolution quotidienne ou mensuelle des dépenses.

- **Filtrage par service** : identifier rapidement si EC2, S3 ou RDS consomment le plus.
- **Filtrage par type d'usage** : distinguer entre stockage, bande passante, snapshots, etc.
- **Filtrage par balises (tags)** : très utile en entreprise pour répartir les coûts par projet, département ou client.
- **Prévisions** : AWS applique des modèles statistiques pour estimer vos coûts à 3 ou 12 mois.

### Exemple d'analyse

- Vous découvrez que vos **snapshots EBS** coûtent 20 % de votre facture → suppression des anciens snapshots inutiles.
- Vous constatez que vos **instances EC2 t3a.micro** en Irlande coûtent plus cher qu'en Virginie → possibilité de migrer vers une région moins coûteuse.

Cost Explorer est **l'outil numéro un** pour identifier les “fuites financières” de votre architecture.

## 4. Stratégies d'optimisation des coûts

### a) Savings Plans (EC2, Lambda, Fargate)

- Engagement sur **1 ou 3 ans**, avec paiement horaire garanti.
- Réduction jusqu'à **60 %** par rapport au prix à la demande.
- Grande flexibilité : un savings plan EC2 peut s'appliquer à différentes tailles d'instances dans une même famille.
- Exemple : au lieu de payer \$17/mois par serveur t3.small, vous réduisez à environ \$10 grâce à un engagement.

Idéal si vous savez que vos serveurs tourneront en continu (site web, API, base de données).

### b) Instances réservées (RDS, EC2)

- Engagement sur **1 à 3 ans** mais moins flexible que les Savings Plans.
- Jusqu'à **70 % de réduction**.
- Exige de conserver le même type d'instance (ex. RDS db.t3.medium).
- Exemple : un serveur MySQL RDS facturé \$0,043/h en mode à la demande tombe à \$0,026/h avec une instance réservée (≈ 40 % d'économie).

Recommandé pour les bases de données critiques qui ne changent pas de configuration fréquemment.

### c) Optimisation du stockage S3

- **Classes de stockage :**
  - **Standard** : accès fréquent, faible latence.
  - **S3 IA (Infrequent Access)** : 40 % moins cher, pour des fichiers rarement utilisés.
  - **S3 Glacier** : stockage d'archives, prix minimal mais accès lent (minutes à heures).
  - **S3 Intelligent-Tiering** : AWS analyse automatiquement vos fichiers et déplace ceux rarement consultés vers une classe moins chère.

Exemple : des backups de base de données quotidiens → après 30 jours, ils passent automatiquement en Glacier.

### d) Auto Scaling & Load Balancing

- **Auto Scaling** : réduit automatiquement le nombre d'instances EC2 en période creuse.
- **Load Balancer** : répartit le trafic pour éviter la surcharge inutile d'un serveur.
- Exemple : un site e-commerce n'a besoin que de 3 serveurs en semaine, mais 10 le week-end. Auto Scaling ajuste automatiquement.

### e) Budgets et alertes

- Fixer un **budget mensuel** (ex. 50 \$).
- Recevoir un mail si les coûts prévisionnels dépassent ce montant.
- Utile pour éviter les mauvaises surprises (surtout lors des tests en environnement gratuit).

## 5. Exemples concrets d'économies

1. **Startup SaaS** : 5 serveurs EC2 t3.medium en continu.
  - Prix à la demande : ≈ \$200/mois.
  - Avec Savings Plan (1 an) : ≈ \$130/mois → **35 % d'économies**.

2. **Site e-commerce** : stockage de 1 To de photos produits.
  - S3 Standard : ≈ \$23/mois.
  - S3 Glacier Flexible Retrieval : ≈ \$4/mois → **80 % d'économies**.
3. **Entreprise avec 50 développeurs** :
  - Attribution de rôles IAM limités + budgets par département.
  - Résultat : réduction de 25 % des coûts liés aux tests inutiles.

## 6. Bonnes pratiques de gestion des coûts

- Toujours **supprimer les ressources non utilisées** (instances, EBS, snapshots).
- Appliquer des **tags obligatoires** (Projet=ClientA, Environnement=Prod) pour mieux suivre les coûts.
- Utiliser **S3 Intelligent-Tiering** pour les fichiers statiques.
- Mettre en place **Auto Scaling** pour EC2.
- Exploiter **Savings Plans et Instances réservées** dès que la consommation est stable.
- Activer **Budgets + Alertes CloudWatch**.
- Vérifier régulièrement **Cost Explorer** pour traquer les anomalies.

## 7. En résumé (Billing & Optimisation)

- **Billing Dashboard** = point central pour surveiller les coûts.
- **Cost Explorer** = outil principal pour analyser et prévoir.
- **Savings Plans** = flexibles, -60 % sur EC2/Lambda/Fargate.
- **Instances réservées** = jusqu'à -70 % sur RDS/EC2 mais plus rigides.
- **Classes de stockage S3** = optimiser entre Standard, IA, Glacier et Intelligent-Tiering.
- **Budgets & alertes** = sécuriser votre facture.
- L'optimisation des coûts est une **compétence clé** pour tout administrateur cloud.

## Glossaire complet des services et concepts AWS (EC2, RDS, S3, IAM, Billing)

Ce glossaire regroupe les principaux termes rencontrés dans le parcours AWS. Chaque définition est accompagnée d'une explication claire et, si possible, d'un exemple concret pour aider à la mémorisation.

### EC2 – Elastic Compute Cloud

- **EC2 (Elastic Compute Cloud)**  
Service d'infrastructure (IaaS) permettant de lancer des machines virtuelles dans le cloud. Ces instances servent à héberger des applications, sites web ou scripts.  
*Exemple* : déployer un site WordPress sur une instance EC2 Ubuntu.
- **Instance EC2**  
Machine virtuelle lancée dans AWS. On choisit son type (CPU, RAM, stockage).  
*Exemple* : une t3.micro pour les tests, une c5.4xlarge pour le calcul intensif.
- **AMI (Amazon Machine Image)**  
Image préconfigurée contenant un système d'exploitation et éventuellement des logiciels.  
*Exemple* : lancer une instance Ubuntu, Windows Server, ou une AMI déjà équipée de WordPress.
- **EBS (Elastic Block Store)**  
Disques durs attachés aux instances EC2. Ils peuvent être sauvegardés via des instantanés.  
*Exemple* : stocker les données d'une base MySQL sur un volume EBS de 20 Go.
- **Snapshot EBS**  
Sauvegarde incrémentielle d'un volume EBS.  
*Exemple* : avant une mise à jour critique, créer un snapshot pour revenir en arrière si nécessaire.
- **Elastic IP**  
Adresse IP publique fixe, assignable à une instance EC2.  
*Exemple* : utile pour héberger un site web accessible via un nom de domaine stable.
- **Security Group**  
Pare-feu virtuel qui définit les règles d'accès (ports, protocoles, adresses IP autorisées).  
*Exemple* : autoriser le port 22 (SSH) uniquement depuis l'adresse IP de l'administrateur.

- **Auto Scaling**

Mécanisme qui ajoute ou supprime automatiquement des instances EC2 selon la charge.

*Exemple* : passer de 3 à 10 serveurs lors d'un pic de trafic e-commerce.

- **Load Balancer (ELB – Elastic Load Balancer)**

Répartit le trafic entre plusieurs instances EC2 pour éviter la surcharge.

*Exemple* : distribuer les requêtes web sur 4 serveurs.

## **RDS – Relational Database Service**

- **RDS (Relational Database Service)**

Service de base de données managée (DBaaS). AWS s'occupe de l'installation, des mises à jour, des sauvegardes et de la haute disponibilité.

*Exemple* : déployer une base MySQL pour un site e-commerce sans configurer de serveur soi-même.

- **Engines disponibles**

- **MySQL** : base relationnelle populaire.
- **MariaDB** : fork open source de MySQL.
- **PostgreSQL** : puissant et extensible.
- **Oracle et SQL Server** : solutions propriétaires.
- **Aurora** : moteur propriétaire d'AWS, compatible MySQL et PostgreSQL, optimisé pour le cloud.

- **Instance RDS**

Machine dédiée uniquement à la base de données.

*Exemple* : une db.t3.micro gratuite pour les tests, une db.r5.large en production.

- **Multi-AZ Deployment**

Réplication synchrone de la base de données dans une autre zone de disponibilité.

*Exemple* : si une zone tombe en panne, la base bascule automatiquement sur la réplique.

- **Snapshots RDS**

Sauvegardes automatiques quotidiennes ou instantanés manuels.

*Exemple* : restaurer une base supprimée par erreur en revenant 3 jours en arrière.

- **Point-in-Time Recovery**

Restauration à une seconde précise dans le passé grâce aux sauvegardes continues.

*Exemple* : corriger une suppression accidentelle de données à 10h42 en restaurant la base à 10h41.

- **Instances réservées RDS**

Engagement sur 1 à 3 ans qui réduit les coûts jusqu'à 70 %.

*Exemple* : base de production critique utilisée en continu.

### **S3 – Simple Storage Service**

- **S3 (Simple Storage Service)**

Service de stockage d'objets (fichiers). Permet de stocker images, vidéos, backups, etc.

*Exemple* : héberger les images produits d'un site e-commerce.

- **Bucket**

Conteneur de stockage S3. Chaque bucket est unique au niveau mondial.

*Exemple* : mes-images-client.

- **Objet**

Fichier stocké dans un bucket, accompagné de métadonnées (taille, auteur, date, etc.).

*Exemple* : photo.jpg avec une métadonnée "auteur = Hugo".

- **Classes de stockage**

- **Standard** : accès fréquent, faible latence.
- **IA (Infrequent Access)** : fichiers peu utilisés.
- **Glacier / Glacier Deep Archive** : archivage à très faible coût, récupération lente.
- **Intelligent-Tiering** : déplace automatiquement les fichiers selon leur usage.

- **Versioning**

Active la conservation de toutes les versions d'un fichier.

*Exemple* : restaurer un fichier écrasé par erreur.

- **Lifecycle Rules (Cycle de vie)**

Automatisations pour déplacer ou supprimer des fichiers.

*Exemple* : supprimer les backups après 180 jours.

- **Bucket Policy**

Stratégie JSON définissant les droits d'accès à un bucket (lecture, écriture, suppression).

*Exemple* : autoriser tout Internet à lire les fichiers dans un dossier public.

- **Sécurité S3**

Par défaut, un bucket est privé. Pour partager des fichiers, il faut configurer IAM ou une bucket policy.

## **IAM – Identity and Access Management**

- **IAM (Identity and Access Management)**

Service de gestion des identités et permissions. C'est le cœur de la sécurité AWS.

*Exemple* : donner à un développeur accès uniquement à S3, mais pas à EC2.

- **Utilisateur IAM**

Identité créée pour une personne (login + mot de passe + clés d'accès).

*Exemple* : alice-dev.

- **Rôle IAM**

Identité temporaire qu'une ressource AWS peut endosser pour obtenir des permissions.

*Exemple* : un serveur EC2 qui téléverse automatiquement des fichiers sur S3.

- **Groupes**

Ensemble d'utilisateurs partageant les mêmes permissions.

*Exemple* : groupe Admins avec accès complet à EC2 et RDS.

- **Politiques IAM**

Documents JSON définissant les permissions. Elles peuvent être gérées par AWS ou personnalisées.

*Exemple* : AmazonS3ReadOnlyAccess ou une politique maison limitant l'accès à un seul bucket.

- **MFA (Multi-Factor Authentication)**

Sécurité renforcée via un deuxième facteur (code SMS, application mobile).

*Exemple* : protéger le compte racine AWS.

## **Billing & Optimisation des coûts**

- **Billing Dashboard**

Tableau de bord central des coûts. Affiche factures, paiements et prévisions.

- **Cost Explorer**

Outil d'analyse visuelle des coûts. Filtrage par service, région, balises.

*Exemple* : voir que 70 % des coûts viennent de snapshots EBS inutiles.

- **Budgets AWS**

Permet de définir un montant maximum et de recevoir des alertes en cas de

dépassement.

*Exemple* : alerte si les coûts dépassent 50 €/mois.

- **Savings Plans**

Engagement de consommation (EC2, Fargate, Lambda) sur 1 ou 3 ans.

Jusqu'à 60 % d'économies.

*Exemple* : site web toujours en ligne → idéal pour des serveurs permanents.

- **Instances réservées (Reserved Instances)**

Réservées à un type d'instance précis (EC2 ou RDS). Jusqu'à 70 % d'économies.

*Exemple* : base MySQL critique qui doit toujours tourner.

- **Classes de stockage S3 économiques**

Utiliser Glacier ou Intelligent-Tiering pour réduire la facture stockage.

- **Bonnes pratiques de réduction des coûts**

- Supprimer les ressources inutiles.
- Utiliser Auto Scaling pour EC2.
- Activer des politiques de cycle de vie sur S3.
- Taguer toutes les ressources pour identifier les coûts par projet.
- Analyser régulièrement Cost Explorer.

## En synthèse

- **EC2** = puissance de calcul flexible.
- **RDS** = base de données managée, facile à utiliser.
- **S3** = stockage d'objets illimité et économique.
- **IAM** = sécurité et gestion des accès centralisée.
- **Billing** = suivi et optimisation des coûts, avec économies possibles de 40 à 70 %.

## Résumé AWS appliqué au métier d'Administrateur Systèmes & DevOps

Ton métier consiste à **concevoir, automatiser, sécuriser et maintenir des infrastructures cloud et hybrides**. Tu dois savoir utiliser les services AWS non pas de manière théorique, mais comme des **briques concrètes** dans tes projets (comme ton projet WordPress + MySQL + Jenkins + Kubernetes sur Azure/AWS). Voici ce

que tu dois **retenir et maîtriser en priorité**, car ce sont les points directement utiles à ton rôle.

## Amazon EC2 – la base du calcul

- **Ce que c'est** : des machines virtuelles flexibles pour héberger applications, middlewares ou services critiques.
- **Lien avec ton métier** : savoir choisir le bon type d'instance (CPU, RAM, coût) selon la charge, les démarrer avec Ansible ou Terraform, et les intégrer dans une architecture complète.
- **À retenir pour toi** :
  - Bien configurer les **Security Groups** (firewall AWS).
  - Gérer la persistance des données avec **EBS** et les snapshots.
  - Fixer les adresses avec les **Elastic IP** si un site doit être stable.
  - Utiliser **Auto Scaling** + **Load Balancer** pour absorber le trafic sans interruption.
- **Dans tes projets** : tu as utilisé des VMs sur Azure/K3s, mais les mêmes principes existent sur EC2 → choisir la bonne instance, sécuriser via SG, et prévoir l'élasticité.

## Amazon RDS – bases de données managées

- **Ce que c'est** : un service DBaaS qui enlève la charge d'installation, patch, sauvegarde et réplication.
- **Lien avec ton métier** : tu dois garantir la disponibilité et la sécurité des bases (WordPress, ERP, monitoring). Avec RDS, tu gagnes du temps et réduis les risques.
- **À retenir pour toi** :
  - **Automatisation des sauvegardes** et restauration "point-in-time".
  - **Multi-AZ** pour haute dispo en production.
  - Bien configurer le **VPC** : DB jamais exposée directement à Internet.
  - Savoir choisir le moteur adapté (MySQL/MariaDB → WordPress, PostgreSQL → ERP, Aurora → haute perf).
- **Dans tes projets** : ton WordPress/MySQL aurait pu tourner sur RDS au lieu de MySQL sur VM, ce qui simplifie la maintenance et augmente la fiabilité.

## Amazon S3 – stockage universel

- **Ce que c'est** : un espace illimité pour stocker objets (fichiers, backups, images, logs).
- **Lien avec ton métier** : stockage des médias WordPress, sauvegardes MySQL, artefacts Jenkins, logs applicatifs, etc.
- **À retenir pour toi** :
  - Organiser tes fichiers dans des **buckets**.
  - Choisir la bonne **classe de stockage** (Standard, Glacier, Intelligent-Tiering).
  - **Versioning** pour éviter la perte de fichiers critiques.
  - **Lifecycle rules** pour automatiser la suppression ou l'archivage (ex : backups > 180 jours → Glacier).
  - Sécurité : ne jamais laisser un bucket public par erreur → utiliser IAM et bucket policies.
- **Dans tes projets** : tu pourrais stocker tes backups WordPress/MySQL ou les images Docker de Jenkins dans S3 pour une persistance simple et fiable.

## IAM – sécurité et gestion des accès

- **Ce que c'est** : le système qui définit **qui fait quoi** dans AWS.
- **Lien avec ton métier** : tu dois garantir la sécurité des accès (zéro partage du compte root, séparation des rôles par projet/équipe).
- **À retenir pour toi** :
  - Créer un **utilisateur IAM par personne** ou service (pas d'accès root en prod).
  - Utiliser **MFA** (double authentification) pour les comptes critiques.
  - Créer des **rôles IAM** pour tes VMs ou pods Kubernetes (ex : autoriser Jenkins à push sur S3 sans clé locale).
  - Comprendre la différence :
    - **IAM Policies** → liées aux utilisateurs/roles.
    - **Bucket Policies / Resource Policies** → liées aux ressources (ex : rendre public un dossier S3).

- **Dans tes projets** : au lieu de gérer des clés SSH ou tokens directement dans tes fichiers, tu pourrais déléguer l'accès via IAM (ex : Jenkins → S3, K3s pods → S3 logs).

## Billing – maîtrise des coûts

- **Ce que c'est** : AWS facture à l'usage ("pay as you go").
- **Lien avec ton métier** : optimiser les coûts fait partie de la mission DevOps → ne pas gaspiller sur des VMs surdimensionnées ou du stockage inutilisé.
- **À retenir pour toi** :
  - Surveiller via **Cost Explorer** pour identifier les postes de dépense.
  - Mettre en place des **budgets et alertes**.
  - Utiliser **Savings Plans / Instances réservées** pour réduire de 40–70 % les coûts si usage continu.
  - Déplacer les backups/archives vers S3 Glacier ou Intelligent-Tiering.
  - Supprimer les ressources inutiles (snapshots, volumes EBS non attachés).
- **Dans tes projets** : toi qui travailles avec des budgets serrés (Azure 170 €, Oracle Free Tier...), cette maîtrise des coûts est un atout clé pour montrer ton sérieux et ton efficacité aux recruteurs.

## Ce que tu dois retenir et appliquer à ton métier

1. **Automatisation et scalabilité** : savoir déployer EC2, RDS, S3 via Ansible/Terraform et intégrer Auto Scaling + Load Balancer.
2. **Sécurité** : IAM est central. Pas de comptes partagés, accès au moindre privilège, MFA obligatoire, politiques bien définies.
3. **Stockage et sauvegardes** : S3 pour fichiers et backups, RDS snapshots pour DB, versioning pour limiter les erreurs humaines.
4. **Optimisation coûts** : surveiller Billing, choisir Savings Plans et classes de stockage adaptées.
5. **Pragmatisme DevOps** : toujours penser résilience (Multi-AZ, snapshots), industrialisation (pipelines CI/CD avec Jenkins), et réduction des interventions manuelles.

En résumé pour toi :

- **EC2** = tes serveurs applicatifs, scalables et sécurisés.
- **RDS** = bases de données fiables, managées, faciles à restaurer.
- **S3** = stockage universel et économique pour médias, backups et artefacts.
- **IAM** = sécurité granulaire, gestion fine des droits et des rôles.
- **Billing** = optimisation financière, savoir présenter aux recruteurs comment tu réduis les coûts.

## Conclusion générale

La maîtrise d’AWS ne se résume pas à connaître quelques services isolés. Pour un administrateur systèmes & DevOps, il s’agit avant tout de comprendre **comment ces services s’intègrent entre eux** et comment ils soutiennent les objectifs principaux d’une infrastructure moderne : **disponibilité, sécurité, automatisation, résilience et optimisation des coûts**.

Dans ton métier, tu dois être capable de concevoir une architecture complète où chaque brique AWS répond à un besoin précis :

- **EC2** t’offre la flexibilité de déployer des serveurs à la demande, tout en t’assurant la possibilité de monter en charge avec l’auto-scaling et de protéger tes services grâce aux security groups et aux IP élastiques.
- **RDS** libère ton équipe de la lourde tâche de gérer une base de données manuellement : plus besoin de t’inquiéter des patchs de sécurité, des sauvegardes quotidiennes ou de la haute disponibilité. Tu peux te concentrer sur la performance et la fiabilité des applications.
- **S3** devient la solution universelle pour stocker de façon simple, scalable et économique tous types de données : médias, backups, fichiers applicatifs, logs. Avec les politiques de cycle de vie et les classes de stockage, tu assures la bonne gestion à long terme tout en réduisant la facture.
- **IAM** agit comme le cœur de la sécurité. C’est grâce à lui que tu peux définir avec précision qui fait quoi, limiter les accès au minimum nécessaire, et assurer la traçabilité de toutes les actions. C’est aussi le garant de la conformité, que ce soit vis-à-vis du RGPD ou des bonnes pratiques de cybersécurité.
- **Billing** et les outils comme **Cost Explorer** te donnent la visibilité et le contrôle indispensables pour éviter les dérives financières. Dans une logique DevOps,

tu ne dois pas seulement livrer rapidement et fiablement, mais aussi garantir que la solution reste économiquement viable.

Ce que tu dois retenir dans une perspective professionnelle, c'est que **ta valeur ajoutée réside dans ta capacité à relier ces services entre eux pour créer des environnements robustes, sécurisés et optimisés**. Un bon DevOps n'est pas seulement un technicien qui sait lancer une instance, mais un architecte qui comprend comment :

- automatiser un déploiement complet via Ansible ou Terraform,
- sécuriser chaque étape grâce à IAM et aux politiques de ressources,
- assurer la continuité de service avec RDS Multi-AZ et S3 versioning,
- anticiper les besoins de montée en charge avec EC2 Auto Scaling et Load Balancer,
- maîtriser les coûts en choisissant les bonnes options de facturation et de stockage.

En pratique, cela veut dire que tu peux présenter à un jury ou à un recruteur non seulement les commandes et procédures que tu maîtrises, mais surtout la **vision globale** : comment tu garantis que l'entreprise gagne en **agilité, en sécurité et en rentabilité** grâce à ton expertise.

En conclusion, AWS t'apporte les briques, mais c'est ta capacité à les assembler intelligemment qui fait de toi un futur administrateur systèmes & DevOps complet. Ton rôle est de transformer des services bruts en une infrastructure cohérente, automatisée et durable, capable d'accompagner la croissance des projets sans jamais compromettre la sécurité ni exploser les coûts. C'est cette approche holistique que tu dois retenir et valoriser dans ta carrière.